DASHBOARD OPERATING GUIDE

NYC 311 / Volume Lab

Python Dash - monthly requests, forecasting, and workforce-cost planning

Prepared 2026-10-24 | Detailed user, data, model, and operational reference

Local URL	http://localhost:8050/
Launch	.venv/bin/python src/app.py
Primary source	Curated Parquet lake plus a live recent NYC Open Data refresh
Main audience	Analysts and supervisors exploring monthly service-request trends

Important data and decision boundary

NYC 311 service-request records are not equivalent to phone-call arrivals, agent schedules, or payroll. Forecast conversions and labor inputs are scenarios unless calibrated against authorized operational records. Dashboard output is planning support and requires human review.

1. Purpose and page layout

The Python Dash application is the monthly analytics and planning dashboard. It lets an analyst compare opened and closed request history, select a borough and analysis window, create a monthly forecast, and estimate monthly call-taker labor cost from configurable assumptions.

The screen has a borough/date filter area, summary indicators, opened-versus-closed history, a forecast section with model and horizon selectors, and a workforce planning view. Charts respond to the corresponding view or optimization action; model and borough choices can differ between the trend and forecast sections.

2. Start-up, data, and refresh

Run the command in the quick-start panel from the repository root. The application reads the curated Parquet partitions for unique_key, created_date, closed_date, and borough, removes invalid creation dates and duplicate request identifiers, and prepares the dashboard's in-memory frame.

At startup, the app refreshes the recent opened/closed daily aggregates from the public NYC Open Data 311 dataset. The page footer reports the latest request date used. A successful refresh updates the in-memory dashboard data; the Parquet cache is not rewritten by the dashboard. A network or source error is surfaced during startup rather than silently relabeling stale data as live.

The source records are service requests, not call-center ACD arrivals, agent timecards, or labor expenditure. Monthly call forecasts and staff costs are planning estimates; calibrate assumptions using operational call-center data before making staffing decisions.

- Set NYC311_DATA_ROOT if the dashboard_data.parquet file or curated lake is stored outside the repository.
- The checked-in daily cache is documented as current through 2026-10-23; a later live refresh may extend the in-memory data.
- The app uses monthly aggregates and fills missing calendar months with zero when constructing a forecast series.

3. Trend analysis

Choose a borough and a date range to inspect opened and closed request counts. The opened series is grouped by creation date; the closed series is grouped by closure date. The two series therefore describe separate lifecycle events and should not be interpreted as a same-cohort completion rate.

The chart is built from daily totals re-aggregated to calendar months. Filtering is applied to the request borough; missing borough labels are represented as Unknown. The monthly display is intended to reveal seasonality, long-run movement, and large changes, not individual case histories.

4. Forecast models and model selection

Available model choices are Best, Holt-Winters, AutoARIMA, and LSTM. The backend requires at least 24 monthly observations for model selection. It reserves a chronological holdout of up to 12 months, evaluates candidates with mean absolute error (MAE), then fits the selected candidate to the full series for the requested horizon.

Holt-Winters uses additive trend and, when enough data exists, annual seasonality. AutoARIMA uses pmdarima automatic order selection with annual seasonality where supported. LSTM is a PyTorch one-variable recurrent model trained on normalized recent monthly values. Negative point forecasts are clipped to zero. Approximate uncertainty bounds widen with forecast horizon; they are not formal calibrated prediction intervals.

Best chooses the candidate with lowest holdout MAE among successfully fitted candidates. Failures are logged and recorded in MLflow; if every candidate fails, the backend raises an error. The result reports the selected model, holdout MAE, and MLflow run identifier.

- For a model-specific result, select that model and press its view action.
- Use longer horizons as exploratory projections; forecast uncertainty and structural change increase with lead time.
- Compare holdout errors as model diagnostics, not as a guarantee of future accuracy.

5. Workforce and labor-cost estimate

The workforce panel turns a selected monthly opened-request forecast into an estimated monthly call volume using the calls-per-request assumption. It estimates required call takers with Erlang-C service-level calculations, applies an occupancy ceiling and paid-shrinkage assumption, and estimates labor spend from wage, operating hours, and business days.

The displayed comparison is forecast-driven staffing versus a fixed-staff baseline equal to peak staffing across the forecast horizon. The difference is a modeled savings estimate. It does not account for intraday call arrival distributions, shift overlap, breaks, skill groups, labor contracts, queue abandonment, or payroll rules.

Editable assumptions include call conversion, average handle time, forecast horizon, hourly wage, shrinkage, daily operating hours, monthly business days, service-level target, answer-time target, occupancy ceiling, borough, and

forecast model. Values are scenario inputs, not measured NYC 311 staffing facts.

6. Interpretation, limitations, and troubleshooting

Treat the charts as decision support. Validate the latest-data date, selected borough, forecast model, holdout error, and every labor assumption before comparing costs. Large or unexpected forecasts should trigger a source and aggregation review.

If the Parquet lake cannot be found, configure NYC311_DATA_ROOT and verify the curated partition paths. If refresh fails, confirm network access to data.cityofnewyork.us and retry. If model fitting fails, inspect the server log and MLflow run record; a short or irregular history may not support every model.

MLflow writes forecast candidate and best-fit runs to the configured tracking store. The default is a local SQLite store under ~/.nyc311_pipeline. See the repository README for tracking URI and artifact-store configuration.